

INF.04 – Podręcznik programisty

Aplikacje konsolowe – IntelliJ IDEA (Java)

Spis treści

- Tworzenie projektu w IntelliJ IDEA (Java)
- Programowanie obiektowe (OOP)
- Dziedziczenie i polimorfizm (słowa kluczowe `this`, `super`)
- Interfejsy i klasy abstrakcyjne
- Wyjątki (`try/catch`, `throw`)
- Kolekcje: `ArrayList`, `HashMap`
- Pliki i strumień wejścia/wyjścia (I/O)
- Wzorce projektowe (Singleton, Factory, Observer, MVC, DAO)
- Testowanie (JUnit)
- REST API (Java HTTP)
- Bazy danych (JDBC, SQLite)
- UML i architektura aplikacji
- Przykłady kodu i zadania egzaminacyjne
- Typowe błędy i debugowanie

Tworzenie projektu w IntelliJ IDEA

Aby rozpocząć pracę z aplikacją konsolową w Javie, należy utworzyć nowy projekt w środowisku IntelliJ IDEA. W IntelliJ wybieramy **File > New > Project**, zaznaczamy typ projektu **Java**, nadajemy mu nazwę (np. `KonsolowaAplikacja`) i ustawiamy lokalizację katalogu. Należy wskazać zainstalowane środowisko Java JDK (lub pobrać je z poziomu IDE) ¹ ². Po utworzeniu projektu IntelliJ wygeneruje dla nas strukturę plików (m.in. folder `src`). Możemy wtedy utworzyć klasę zawierającą metodę `public static void main(String[] args)`. Przykład minimalnego kodu:

```
public class Main {  
    public static void main(String[] args) {  
        System.out.println("Witaj, świecie!");  
    }  
}
```

Program ten po uruchomieniu w konsoli wyświetli napis „Witaj, świecie!”.

Konfiguracja projektu: Najnowsze wersje IntelliJ IDEA (Community lub Ultimate) zawierają wbudowane wsparcie dla projektów Java. Podczas tworzenia projektu możemy też wybrać system budowania (np. Maven lub Gradle), choć na potrzeby konsolowych programów proste

rozwiązanie „plain Java” w zupełności wystarczy ¹. Upewnij się, że wersja JDK jest zgodna z wymaganiami (np. Java 8, 11, 17, zależnie od matury lub wytycznych INF.04).

Programowanie obiektowe (OOP)

Aplikacje Java są zazwyczaj budowane w paradygmacie programowania obiektowego (OOP). W OOP program dzielimy na **klasy** (szablony danych i zachowań) i **obiekty** (egzemplarze tych klas). Każda klasa może posiadać pola (zmienne instancji) oraz metody (funkcje). Stosujemy **enkapsulację** – pola klasy zazwyczaj są `private`, a dostęp do nich zapewniają metody `public` (getter i setter). Klasa może również mieć metody `public`, które opisują jej zachowania. Dzięki OOP mamy **polimorfizm** – ten sam kod może działać na obiektach różnych klas, jeśli dziedziczą one po wspólnym typie lub implementują ten sam interfejs. Na przykład:

```
public class Zwierze {
    public void dajGlos() {
        System.out.println("Głos zwierza");
    }
}

public class Pies extends Zwierze {
    @Override
    public void dajGlos() {
        System.out.println("Hau hau");
    }
}

public class Kot extends Zwierze {
    @Override
    public void dajGlos() {
        System.out.println("Miau");
    }
}
```

W powyższym kodzie zarówno `Pies`, jak i `Kot` dziedziczą po klasie `Zwierze`, ale implementują metodę `dajGlos()` na swój sposób. Jeśli utworzymy `Zwierze z = new Pies(); z.dajGlos();`, wykona się metoda z klasy `Pies`. To jest polimorfizm w działaniu.

Kluczowe elementy OOP to: **abstrakcja** (modelowanie problemu w klasach), **enkapsulacja** (ukrywanie implementacji wewnątrz klas), **dziedziczenie** (rozszerzanie klas bazowych) i **polimorfizm** (jedna metoda – różne realizacje w podklasach). W Javie wszystkie klasy dziedziczą po klasie `Object`, co zapewnia mechanizmy takie jak `equals()`, `hashCode()`, `toString()` itp.

Dziedziczenie i polimorfizm (słowa kluczowe `this`, `super`)

W Javie klasa może **dziedziczyć** po jednej klasie bazowej (ograniczenie dziedziczenia jednokrotnego). Służy do tego słowo kluczowe `extends`. Przy dziedziczeniu podklasa przejmie wszystkie pola i metody klasy bazowej (poza konstruktorami). Przykładowo:

```
public class Pracownik {
    private String imie;
    private String nazwisko;

    public Pracownik(String imie, String nazwisko) {
        this.imie = imie;
        this.nazwisko = nazwisko;
    }

    public String getImie() { return imie; }
    public String getNazwisko() { return nazwisko; }
}

public class Lekarz extends Pracownik {
    private double premia;

    public Lekarz(String imie, String nazwisko, double premia) {
        super(imie, nazwisko); // wywołanie konstruktora klasy bazowej
        this.premia = premia;
    }

    public double getPremia() { return premia; }
}
```

W powyższym przykładzie `Lekarz` dziedziczy po klasie `Pracownik`. Słowo kluczowe `super` pozwala wywołać konstruktor (lub metodę) klasy bazowej o takich samych parametrach ³. Dzięki temu nie musimy ponownie przypisywać pól `imie` i `nazwisko` – wystarczy wywołać `super(imie, nazwisko)`. W kodzie `Lekarz` korzysta z odziedziczonych metod `getImie()` i `getNazwisko()` bez dodatkowej implementacji.

Słowo kluczowe `this` odwołuje się do bieżącego obiektu (instancji klasy) i umożliwia rozróżnienie zmiennych pola od parametrów metody/konstruktora ⁴. Przykład użycia `this` w konstruktorze:

```
public class Punkt {
    private int x, y;

    public Punkt(int x, int y) {
        this.x = x; // this.x to pole klasy, x to parametr
        this.y = y;
    }
}
```

```
}
}
```

Bez `this` odwoływalibyśmy się do lokalnych parametrów metody, więc użycie `this` gwarantuje modyfikację właściwości obiektu. Podobnie `super` używamy w metodach, np. `super.metoda()` wywoła implementację klasy bazowej (inną niż wywoływana przez instancję) ³.

Dzięki dziedziczeniu i polimorfizmowi możemy pisać bardziej elastyczny kod. Przykładowo listę obiektów typu `Zwierze` (z wcześniejszego przykładu) możemy wypełnić różnymi podklasami (`Pies`, `Kot`) i wywoływać wspólne metody bez sprawdzania typu w kodzie.

Interfejsy i klasy abstrakcyjne

Java pozwala również definiować **interfejsy** oraz **abstrakcyjne klasy**, które określają „szkielet” zachowania.

- **Klasa abstrakcyjna** to klasa, której nie można bezpośrednio instancjonować. Może zawierać zarówno metody zaimplementowane, jak i **metody abstrakcyjne** (deklaracje bez ciała). Podklasy dziedziczą po niej wszystkie zdefiniowane metody i muszą implementować metody abstrakcyjne. Przykład klasy abstrakcyjnej i podklasy:

```
public abstract class Kształt {
    public abstract double obliczPole();
}
public class Koło extends Kształt {
    private double promień;
    public Koło(double promień) { this.promień = promień; }
    @Override
    public double obliczPole() { return Math.PI * promień * promień; }
}
```

W tym przykładzie `Koło` dziedziczy po abstrakcyjnej klasie `Kształt` i implementuje metodę `obliczPole()`.

- **Interfejs** to rodzaj kontraktu: może zawierać deklaracje metod (od Javy 8 również *metody domyślne* `default` i metody `static` z implementacją). Interfejsy nie przechowują stanu (brak zwykłych pól, tylko stałe `static final`). Klasa **implementująca** interfejs musi zaimplementować wszystkie jego metody abstrakcyjne ⁵. Przykład interfejsu i implementacji:

```
public interface Latajacy {
    void lec();
}
public class Ptak implements Latajacy {
    @Override
    public void lec() {
        System.out.println("Ptaszysko leci!");
    }
}
```

```
}
}
```

Poniższa tabela porównuje najważniejsze cechy klasy abstrakcyjnej i interfejsu w Javie:

Cecha	Klasa abstrakcyjna	Interfejs
Tworzenie instancji	Nie można tworzyć obiektów klasy abstrakcyjnej	Nie można tworzyć obiektów interfejsu
Pola (stan)	Może mieć pola (zmienne instancji)	Od Javy 8 tylko <code>static final</code> (stałe)
Metody	Może zawierać metody z kodem i metody abstrakcyjne	Zawiera deklaracje metod (od J8 także <code>default</code> z ciałem)
Dziedziczenie/ wielo-	Klasa dziedziczy po jednej klasie abstrakcyjnej	Klasa może implementować wiele interfejsów
Główne przeznaczenie	Współdzielenie kodu między klasami oraz hierarchia	Definiowanie kontraktów zachowania (bez implementacji)

Z punktu widzenia wzorców projektowych, klasy abstrakcyjne używa się gdy chcemy wymusić pewną wspólną implementację lub hierarchię, natomiast interfejsy służą do deklarowania zachowań, które mogą być dodane do dowolnej klasy ⁵.

Wyjątki (try/catch, throw)

W Javie mechanizm obsługi błędów i wyjątków jest dobrze rozwinięty. **Wyjątek** (exception) to obiekt sygnalizujący błąd podczas wykonywania programu. Możemy łapać wyjątki i obsługiwać je za pomocą konstrukcji `try-catch-finally` ⁶. Ogólny schemat wygląda tak:

```
try {
    // kod mogący spowodować wyjątek
} catch (TypWyjątku1 e) {
    // obsługa błędu typu TypWyjątku1
} catch (TypWyjątku2 e) {
    // obsługa innego typu wyjątku
} finally {
    // blok wykona się niezależnie od tego, czy wystąpił wyjątek
}
```

Przykład: wczytanie elementu z tablicy może spowodować `ArrayIndexOutOfBoundsException`. Zabezpieczamy się:

```
int[] tab = {1,2,3};
try {
```

```

    int value = tab[5];
    System.out.println(value);
} catch (ArrayIndexOutOfBoundsException ex) {
    System.out.println("Błędny indeks tablicy: " + ex.getMessage());
} finally {
    System.out.println("Zakończono próbę odczytu z tablicy.");
}

```

Jeśli wystąpi wyjątek, kod w bloku `catch` go przechwytuje. Możemy też samodzielnie zgłosić wyjątek przy pomocy `throw new ExceptionType("opis")`. Warto pamiętać, że niektóre wyjątki są **sprawdzone** (checked) – np. `IOException` – wymagają zadeklarowania ich w sygnaturze metody lub przechwycenia w `try-catch`, natomiast inne są **niesprawdzone** (unchecked), jak np. `RuntimeException` (np. `NullPointerException` czy `ArrayIndexOutOfBoundsException`). Rzucając wyjątek z metody, używamy instrukcji `throws` w deklaracji metody lub łapiemy je wewnątrz metody.

Debugowanie wyjątków: Podczas wykonywania programu, gdy wystąpi wyjątek, JVM wypisuje ślad stosu (stack trace) na konsolę. Kluczowe jest odczytanie linijki, która zawiera nazwę wyjątku (np. `java.lang.NullPointerException`) i ścieżkę klasy/metody, gdzie wystąpił błąd. W ten sposób lokalizujemy problem i poprawiamy kod (np. sprawdzając warunki przed dostępem do elementów lub zabezpieczając kod `if`, albo używając `try-catch`).

Kolekcje: `ArrayList`, `HashMap`

Java oferuje bogaty **Collections Framework**, czyli zestaw klas do przechowywania grup obiektów. Najczęściej używanymi kolekcjami są:

- **List** (lista) – uporządkowany zbiór elementów, mogą zawierać duplikaty. Najpopularniejsze implementacje to `ArrayList` i `LinkedList`. `ArrayList` to dynamiczna tablica, która automatycznie zmienia rozmiar ⁷. Przykład:

```

List<String> miasta = new ArrayList<>();
miasta.add("Warszawa");
miasta.add("Kraków");
System.out.println(miasta.get(0)); // Warszawa
System.out.println(miasta.contains("Kraków")); // true

```

`ArrayList` udostępnia metody `add()`, `get(int)`, `remove()`, `size()`, `contains()`, itp. Dodawanie elementu (`.add`) amortyzowane jest zazwyczaj w czasie stałym. Dostęp przez indeks (`.get`) działa w stałym czasie.

- **Set** (zbiór) – przechowuje **unikalne** elementy (bez duplikatów). Implementacjami są m.in. `HashSet`, `TreeSet`.

- **Map** – przechowuje pary klucz-wartość. Najczęściej używaną implementacją jest `HashMap` ⁸.
Przykład:

```
Map<Integer, String> mapa = new HashMap<>();
mapa.put(1, "Jeden");
mapa.put(2, "Dwa");
System.out.println(mapa.get(2)); // Dwa
```

`HashMap` umożliwia szybki (średnio $O(1)$) dostęp do wartości po kluczu. Inne przydatne metody to `remove(key)`, `containsKey(key)`, `keySet()`, `values()`.

Ogólnie, **kolekcje** charakteryzują się dużą elastycznością (rozmiar zmienia się dynamicznie) i wiele gotowych metodami do operacji na zbiorach danych ⁹. Używanie generyków (`List<T>`, `Map<K, V>`) zapewnia bezpieczeństwo typów i wczesne wykrywanie błędów. Przykładowo `ArrayList<String>` przechowuje tylko `String`, co zapobiega błędom castingu.

Typowe operacje: Dodawanie elementu: `list.add(obj)`. Iterowanie: za pomocą pętli `for` lub iteratora. Usuwanie: `list.remove(index)` lub `list.remove(obj)`. W przypadku `Map`: `map.get(key)`, `map.put(key, value)`.

Pliki i strumień wejścia/wyjścia (I/O)

Zapis i odczyt danych z plików to częsty element zadań egzaminacyjnych. Java udostępnia klasy z pakietu `java.io` oraz `java.nio` do pracy z I/O. Podstawowym mechanizmem są **strumienie**: `InputStream` i `OutputStream` (dla bajtów) oraz `Reader` i `Writer` (dla znaków). Przykład odczytu z pliku tekstowego:

```
try (InputStream input = new FileInputStream("dane.txt")) {
    int znak;
    while ((znak = input.read()) != -1) {
        System.out.print((char) znak);
    }
} catch (IOException e) {
    System.out.println("Błąd pliku: " + e.getMessage());
}
```

Powyższy przykład używa **try-with-resources**, co automatycznie zamyka strumień. W Java bez tego należałoby jawnie wywołać `input.close()` w bloku `finally` ¹⁰ ¹¹. Klasa `FileInputStream` może zgłosić `FileNotFoundException` – wyjątek sprawdzany, dlatego obejmujemy kod blokiem `try-catch` ¹⁰. Aby zapisać dane, można użyć `FileOutputStream`:

```
try (OutputStream output = new FileOutputStream("wynik.txt")) {
    String zawartość = "Przykładowy tekst";
    byte[] bajty = zawartość.getBytes("UTF-8");
    output.write(bajty);
}
```

```

} catch (IOException e) {
    System.out.println("Błąd zapisu: " + e.getMessage());
}

```

Tutaj zamykamy strumień automatycznie w `finally`. W przykładach `FileInputStream` i `FileOutputStream` odwołujemy się do plików bajtowo. Możemy też użyć strumieni znakowych (`FileReader`, `BufferedReader`, `FileWriter` itp.) dla wygodniejszej obsługi tekstu. Kluczowe zasady: **zawsze zamykać strumień** (lub używać try-with-resources), obsługiwać wyjątki i zawsze sprawdzać, czy plik faktycznie istnieje (lub utworzyć go przed zapisem).

Wzorzec Dekorator: Wiele klas I/O w Javie implementuje **wzorzec dekoratora** – można „owinać” strumień podstawowy kolejnym strumieniem, np. `BufferedInputStream` opakowuje `FileInputStream` dla efektywniejszego odczytu ¹⁰.

Wzorce projektowe (Singleton, Factory, Observer, MVC, DAO)

Wzorce projektowe to sprawdzone rozwiązania typowych problemów w projektowaniu oprogramowania ¹². Do INF.04 warto znać przynajmniej kilka:

- **Singleton** – gwarantuje istnienie tylko jednej instancji danej klasy. Przydaje się np. do zarządzania konfiguracją lub połączeniem z bazą. W Java implementacja polega na prywatnym konstruktorze i statycznej metodzie dostępowej. Przykład:

```

public class DatabaseConnection {
    private static DatabaseConnection instance;
    private DatabaseConnection() { /* inicjalizacja */ }
    public static synchronized DatabaseConnection getInstance() {
        if (instance == null) {
            instance = new DatabaseConnection();
        }
        return instance;
    }
}

```

Cel wzorca Singleton opisano jako zapewnienie „jednego obiektu danej klasy” ¹³.

- **Factory Method (metoda fabryczna)** – pozwala na tworzenie obiektów bez ujawniania dokładnej klasy tworzonego obiektu. Klasa bazowa posiada fabryczną metodę, a podklasy decydują, jaką konkretną instancję utworzyć ¹⁴. Przykład: klasa `Twórca` definiuje metodę `stworzProdukt()`, a klasy dziedziczące zwracają różne implementacje `Produkt`. Factory Method sprzyja elastyczności i spełnia zasadę otwarte/zamknięte (możemy dodawać nowe klasy produktów bez modyfikacji fabryki) ¹⁴.

- **Observer (Obserwator)** – pozwala na powiadamianie wielu obiektów o zdarzeniach w innym obiekcie. Typowe zastosowanie: model powiadamia interfejsy (widoki) o zmianie danych. W Javie przykładem jest `java.util.Observer` (przestarzały) lub biblioteki takich jak RxJava.
- **MVC (Model-View-Controller)** – wzorzec architektoniczny rozdzielający **logikę (model)** od **interfejsu użytkownika (widok)** i **kontrolera** sterującego przepływem. Widok przyjmuje dane od modelu, a kontroler reaguje na zdarzenia (np. kliknięcia) i modyfikuje model. Dzięki MVC kod staje się czystszy i łatwiejszy w utrzymaniu. Opis wzorca: MVC separuje logikę biznesową od warstwy prezentacji ¹⁴.
- **DAO (Data Access Object)** – wzorzec dostępu do danych ¹⁵. DAO jest obiektem udostępniającym jednolity interfejs do komunikacji z bazą danych lub innym źródłem danych. Dzięki DAO logika aplikacji (model) nie zna szczegółów implementacji przechowywania danych. To umożliwia łatwą wymianę źródła danych (np. przejście z pliku na bazę SQL) bez zmiany kodu logiki biznesowej. Wzorzec DAO jest często stosowany razem z MVC, aby oddzielić dostęp do danych od kontrolera i widoku ¹⁵.

Podsumowując: użycie wzorców projektowych poprawia czytelność i skalowalność kodu ¹². Np. Singleton ogranicza klasę do jednej instancji ¹³, fabryka oddziela tworzenie obiektów od kodu klienta, a MVC dzieli aplikację na warstwy.

Testowanie (JUnit)

Testowanie to ważna część programowania. **Testy jednostkowe** w Javie najczęściej pisze się z użyciem frameworku JUnit. Test jednostkowy to krótka metoda sprawdzająca określoną funkcjonalność. Dobrą praktyką jest pisać wiele małych testów – każdy testuje jedną prostą rzecz ¹⁶.

W JUnit 5 tworzymy klasę testową z metodami oznaczonymi adnotacją `@Test`. Przykład:

```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class KalkulatorTest {

    @Test
    public void testDodawanie() {
        Kalkulator k = new Kalkulator();
        assertEquals(5, k.dodaj(2, 3));
    }

    @Test
    public void testDzieleniePrzezZero() {
        Kalkulator k = new Kalkulator();
        assertThrows(ArithmeticException.class, () -> k.dziel(10, 0));
    }
}
```

```
}
}
```

Pierwszy test sprawdza, czy metoda `dodaj(2, 3)` zwraca `5`. Drugi test oczekuje wyjątku `ArithmeticException` przy dzieleniu przez zero. Jeśli implementacja `Kalkulator` jest poprawna, testy przejdą bez błędów. **JUnit** automatyzuje uruchamianie testów i raportowanie rezultatów. Zalecane jest także praktykowanie TDD (Test-Driven Development): najpierw napisz testy, potem implementację.

Narzędzia do testowania: Poza JUnit, można użyć biblioteki Mockito do mockowania obiektów (symulowania zależności) lub frameworków do testów integracyjnych. W aplikacjach mobilnych istnieją testy instrumentacyjne (Espresso), ale do INF.04 wystarczy znajomość JUnit.

REST API (Java HTTP)

Aplikacje konsolowe mogą komunikować się z siecią (np. łączyć się z serwisem WWW). W Javie od wersji 8 klasy `java.net.HttpURLConnection` pozwalają na realizację prostych żądań HTTP (GET/POST) ¹⁷. Przykładowo, aby pobrać dane z adresu URL:

```
try {
    URL url = new URL("http://example.com/api/dane");
    HttpURLConnection conn = (HttpURLConnection) url.openConnection();
    conn.setRequestMethod("GET");
    int code = conn.getResponseCode();
    if (code == HttpURLConnection.HTTP_OK) {
        InputStream in = conn.getInputStream();
        // odczytaj dane np. przy pomocy Scanner lub BufferedReader
    }
    conn.disconnect();
} catch (IOException e) {
    System.out.println("Błąd HTTP: " + e.getMessage());
}
```

Klasa `HttpURLConnection` udostępnia metodę `getResponseCode()`, a następnie strumień odpowiedzi do odczytu. Więcej na ten temat pokazuje dokumentacja: „`HttpURLConnection` z pakietu `java.net` służy do wysyłania żądań HTTP GET i POST” ¹⁷. W praktyce kod najczęściej sparsuje odpowiedź JSON lub XML i zamieni na obiekty w Javie. Biblioteki takie jak Apache HttpClient lub (w Androidzie) Retrofit/OkHttp ułatwiają pracę z REST, ale w czystej Javie `HttpURLConnection` wystarczy.

Bazy danych (JDBC, SQLite)

Często w zadaniach INF.04 pojawiają się bazy danych. W Javie standardowo używa się **JDBC** (Java Database Connectivity) do łączenia z relacyjną bazą. Jeśli zadanie wymaga prostej bazy plikowej, często korzysta się ze **SQLite**. Aby użyć SQLite w Javie, wystarczy dołączyć sterownik JDBC (np. „SQLite JDBC Driver” z projektu

Xerial) ¹⁸. SQLite nie wymaga instalacji serwera (zapisuje dane w pliku) i jest na licencji publicznej ¹⁹.
Przykładowy kod:

```
// Załaduj sterownik
Class.forName("org.sqlite.JDBC");
Connection conn = DriverManager.getConnection("jdbc:sqlite:baza.db");
Statement stat = conn.createStatement();
stat.executeUpdate("CREATE TABLE IF NOT EXISTS Osoba (id INTEGER PRIMARY KEY,
imie TEXT)");
stat.executeUpdate("INSERT INTO Osoba (imie) VALUES ('Jan')");
ResultSet rs = stat.executeQuery("SELECT * FROM Osoba");
while (rs.next()) {
    System.out.println("Osoba: " + rs.getString("imie"));
}
rs.close();
conn.close();
```

Warto korzystać z klas DAO (wzorec Data Access Object), które zamykają szczegóły JDBC w metodach dostępu do danych. Dzięki JDBC aplikacja na Windows/Linux/Mac może obsługiwać dowolną bazę (po zmianie sterownika i URL).

SQLite w Javie: JavaStart tłumaczy, że SQLite jest idealna do przechowywania danych w aplikacjach, ponieważ nie wymaga serwera i jest bardzo lekka ¹⁹ ¹⁸. Sterownik Xerial (JDBC) pozwala w pełni wykorzystać SQL (CREATE TABLE, INSERT, SELECT itp.) w Javie, analogicznie jak w MySQL czy PostgreSQL.

W przypadku zadań aplikacji konsolowej INF.04 wystarczy najczęściej użyć SQLite lub wbudowanej bazy H2/Derby. Klasy JDBC to `Connection`, `Statement`/`PreparedStatement`, `ResultSet`. Pamiętaj o obsłudze wyjątków SQL i zamykaniu połączenia (najlepiej w `finally` lub przy pomocy try-with-resources).

UML i architektura aplikacji

Przed pisanem kodu warto zaprojektować aplikację. **UML (Unified Modeling Language)** to zunifikowany język modelowania, składający się z diagramów umożliwiających formalny opis struktur i procesów ²⁰. Najpopularniejsze diagramy UML to diagram klas (klasy i ich atrybuty/metody), diagram sekwencji (kolejność wywołań) oraz diagram przypadków użycia. Na egzaminie INF.04 często wymaga się narysowania prostego diagramu klas – ważne jest pokazanie klas, ich relacji (dziedziczenie, asocjacje) oraz kluczowych metod/atributów.

Podstawowe elementy UML: - **Klasa** – przedstawiana jako prostokąt z trzema sekcjami (nazwa, atrybuty, operacje). - **Dziedziczenie** – strzałka z pustą grotą do góry (klasa podrzędna → nadklasa). - **Asocjacja/zależność** – linie między klasami (np. klasa `A` ma referencję do klasy `B`). - **Interfejs** – oznaczany jako stereotyp `«interface»` lub pierścień.

Modelując aplikację, warto też przemyśleć architekturę warstwową (np. podział na **model** (logika + DAO), **widok** (w konsoli – interfejs tekstowy) i ewentualnie **kontroler**).

Diagram klas: Najczęściej wymagany do pokazania struktur danych. Na przykład można narysować klasę `Osoba`, klasę `Samochód` i relację „posiada” (osoba posiada listę samochodów) lub **dziedziczenia** (np. `Student` dziedziczy po `Osoba`). Diagram ma pomóc w zrozumieniu związków między klasami przed implementacją.

Przykłady kodu i zadania egzaminacyjne

Przykład 1: Szyfr Cezara

Zadanie: Utworzyć program, który szyfruje i odszyfrowuje tekst przy pomocy **szyfru Cezara** (przesunięcie liter oadaną liczbę miejsc w alfabecie).

Opis: Użytkownik podaje tekst i klucz (liczbę przesunięcia). Program wyświetla zaszyfrowany tekst. Następnie umożliwia odszyfrowanie.

Implementacja (fragment):

```
public class SzyfrCezara {
    public static String szyfruj(String tekst, int klucz) {
        StringBuilder wynik = new StringBuilder();
        for (char c : tekst.toCharArray()) {
            if (Character.isLetter(c)) {
                char base = Character.isUpperCase(c) ? 'A' : 'a';
                char zaszyfr = (char)((c - base + klucz) % 26) + base;
                wynik.append(zaszyfr);
            } else {
                wynik.append(c);
            }
        }
        return wynik.toString();
    }

    public static String odszyfruj(String tekst, int klucz) {
        return szyfruj(tekst, 26 - (klucz % 26));
    }

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Podaj tekst: ");
        String tekst = scanner.nextLine();
        System.out.print("Podaj klucz (0-25): ");
        int klucz = scanner.nextInt();
        String szyfrogram = szyfruj(tekst, klucz);
        System.out.println("Zaszyfrowany tekst: " + szyfrogram);
        System.out.println("Odszyfrowanie: " + odszyfruj(szyfrogram, klucz));
    }
}
```

Analiza: Funkcja `szyfruj` przesuwa litery w alfabecie, zachowując wielkość liter. Jeżeli `tekst` zawiera niealfabetyczne znaki (np. spacje, cyfry), są kopiowane bez zmian. Odszyfrowanie realizujemy poprzez przesunięcie o `26 - (klucz mod 26)`. Program powinien prawidłowo działać dla alfabetu łacińskiego (26 liter). Potencjalne błędy: jeśli użytkownik poda nie-liczbowy klucz, wystąpi `InputMismatchException`; należałoby wtedy złapać wyjątek lub dodać dodatkową walidację.

Przykład 2: Książka i biblioteka

Zadanie: Napisz aplikację konsolową, która zarządza prostym katalogiem biblioteki. Umożliwia dodawanie książek (`tytuł`, `autor`, `ISBN`), wyświetlanie listy książek oraz zapisywanie/odczytywanie ich z pliku.

Rozwiązanie: - Utwórz klasę `Ksiazka` z polami `String tytuł`, `autor`, `isbn` i metodą `toString()`.

- W `main` trzymaj `List<Ksiazka> biblioteka = new ArrayList<>()`.

- Dodawanie: wczytaj dane, utwórz obiekt `Ksiazka` i dodaj do listy.

- Zapis do pliku (np. CSV): w pętli iteruj listę i zapisuj dane `title;author;isbn\n`.

- Odczyt z pliku: wczytaj każdy wiersz, rozdziel `split(";")`, zbuduj obiekt `Ksiazka` i dodaj do listy.

Przykładowy fragment zapisu:

```
try (PrintWriter writer = new PrintWriter("biblioteka.txt")) {
    for (Ksiazka k : biblioteka) {
        writer.println(k.getTytuł() + ";" + k.getAutor() + ";" + k.getIsbn());
    }
} catch (FileNotFoundException e) {
    System.out.println("Nie można zapisać pliku: " + e.getMessage());
}
```

Odczyt z pliku analogicznie z `Scanner` i `nextLine()`, a następnie `String[] dane = line.split(";")`.

Analiza: Zadanie łączy obiektowe podejście (klasa `Ksiazka`), obsługę kolekcji (`ArrayList`) i operacje I/O. Trzeba także zadbać o wyjątki (plik może nie istnieć) oraz poprawność formatu pliku. Poprawność kodu można sprawdzić testowo: dodaj kilka książek ręcznie, wyświetl listę, zapisz i wczytaj ponownie, upewniając się, że dane się zgadzają.

Typowe błędy i debugowanie

- **NullPointerException:** Pojawia się, gdy odwołujemy się do metody lub pola obiektu, który jest `null`. Np. niezainicjowana lista lub `Scanner`, a próbujemy wykonać na niej operację. Debug: sprawdź, czy obiekt został utworzony (`new`) przed użyciem.
- **ArrayIndexOutOfBoundsException:** Błąd wyjścia poza zakres tablicy/listy. Zawsze sprawdzaj, czy indeks jest poprawny ($0 \leq \text{indeks} < \text{rozmiar}$). Używaj `list.size()`.
- **FileNotFoundException:** Nieprawidłowa ścieżka pliku. Sprawdź ścieżkę względną/absolutną i uprawnienia.

- **Syntax Error (błąd składni):** Np. brakujący średnik `;`, klamry `{}`, czy literówka w nazwie klasy/metody. Kompilator wskazuje linię błędu – popraw literówkę lub dopisz brakujący znak.
- **Logical error:** Program kompiluje się, ale wynik jest niepoprawny. Wtedy przydaje się debugowanie krok po kroku (debugger) lub wypisywanie do konsoli. Przyjrzyj się warunkom, pętlom, czy dane wejściowe są prawidłowo przetwarzane.

Podpowiedź: Zawsze czytaj dokładnie komunikaty błędów Java – wskazują one typ wyjątku i miejsce wystąpienia. Śledź logikę programu w najprostszym przypadku (testowanie z małą liczbą danych) i stopniowo rozbudowuj.

Aplikacje mobilne – Android Studio

Spis treści

- Tworzenie projektu w Android Studio (Java)
- Podstawy Android (Activity, UI, layouty)
- Cykl życia Activity i wielowątkowość
- Interfejs użytkownika i nawigacja
- Bazy danych w Android (SQLite, Room)
- Wzorce projektowe w aplikacjach mobilnych
- Testowanie aplikacji Android
- REST API (wykorzystanie sieci w aplikacji mobilnej)
- UML i struktura aplikacji Android
- Przykłady kodu i zadania egzaminacyjne
- Typowe błędy i debugowanie w Androidzie

Tworzenie projektu w Android Studio

Aby rozpocząć, otwórz Android Studio i wybierz **File > New > New Project**. W kreatorze nowego projektu wybierz typ szablonu (np. **Empty Activity**), nazwij aplikację (np. `BibliotekaApp`), podaj `Package name`, miejsce zapisu i wybierz **Language: Java** ²¹. Konfiguruj minimalny **API Level** (np. API 21 Lollipop lub wyżej). Android Studio utworzy strukturę projektu z plikami `MainActivity.java`, `activity_main.xml` i innymi. Projekt bazuje na Gradle – pliki `build.gradle` (poziom aplikacji i modułu) zawierają zależności.

Konfiguracja: Nowoczesne Android Studio (Arctic Fox i nowsze) automatycznie instaluje wymagane SDK. Upewnij się, że masz zainstalowane przynajmniej Android SDK Platform oraz emulator lub podłączone urządzenie do testów.

Podstawy Android (Activity, UI, layouty)

Aplikacja Androidowa składa się z **aktywnych ekranów** zwanych `Activity`. Każde `Activity` jest klasą dziedziczącą po `android.app.Activity` (lub `AppCompatActivity`) i ma plik układu (layout) określający interfejs użytkownika. Przykład prostego `Activity`:

```

public class MainActivity extends AppCompatActivity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main); // ładowanie układu z pliku XML
        Button btn = findViewById(R.id.myButton);
        btn.setOnClickListener(view -> {
            Toast.makeText(this, "Kliknięto!", Toast.LENGTH_SHORT).show();
        });
    }
}

```

Plik `activity_main.xml` (w folderze `res/layout`) może zawierać definicję UI w języku XML, np. przycisk:

```

<LinearLayout ... >
    <Button
        android:id="@+id/myButton"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Kliknij mnie!" />
</LinearLayout>

```

Interfejs buduje się poprzez **widoki (View)**: `TextView`, `Button`, `EditText`, `ListView` itp. Do każdego widoku możemy się odwołać w kodzie za pomocą `findViewById(R.id.myView)`. Pamiętaj, że identyfikatory w XML (np. `@+id/myButton`) muszą odpowiadać użytym w `findViewById`.

Cykl życia Activity i wielowątkowość

Ważną cechą Androida jest zarządzanie cyklem życia `Activity`. Metody takie jak `onCreate()`, `onStart()`, `onResume()`, `onPause()`, `onStop()`, `onDestroy()` są wywoływane w odpowiednich momentach (tworzenie, wznawianie, zatrzymywanie itp.). Np. należy inicjalizować UI w `onCreate()`, a zwalniać zasoby w `onDestroy()`. Dokumentacja Androida dokładnie opisuje te etapy.

Aplikacje Android wymagają ostrożności przy operacjach długotrwałych – np. zapytania sieciowe lub odczyt z bazy nie mogą być wykonywane na **wątku głównym (UI Thread)**, bo zablokują interfejs i spowodują `Application Not Responding (ANR)`. Dlatego stosujemy różne mechanizmy: `AsyncTask` (starsze API), `Thread` / `Runnable`, czy biblioteki `ExecutorService`, a w nowszych projektach często `Coroutines` (Kotlin) lub RxJava. W zadaniach INF.04 wystarczy wspomnieć konieczność wykonywania takich operacji asynchronicznie (np. przy użyciu `AsyncTask` w Javie lub nowego `java.util.concurrent`).

Interfejs użytkownika i nawigacja

Tworzenie UI w Androidzie odbywa się głównie w XML (layouty) lub programowo. Używamy `Activity` jako ekranów. Do przełączania się między ekranami służą **intenty**. Przykład otwarcia nowej aktywności:

```
Intent intent = new Intent(MainActivity.this, DrugaActivity.class);
intent.putExtra("klucz", 123);
startActivity(intent);
```

W Androidzie obowiązuje też **odsluchiwanie zdarzeń** poprzez nadpisywanie callbacków: np. kliknięcie przycisku `Button` obsługujemy za pomocą `setOnClickListener`. Inny popularny mechanizm to `RecyclerView` (lista elementów) i `Adapter`.

Dla interfejsu mobilnego ważne jest również korzystanie z **zasobów** (strings, kolory, dimsens) – znajdują się w katalogach `res/values`. Dzięki temu łatwiej jest tłumaczyć tekst lub modyfikować UI w jednym miejscu.

Bazy danych w Android (SQLite, Room)

Android oferuje wbudowane wsparcie dla baz SQLite. Mechanizm jest podobny do klasycznego JDBC, ale z abstrakcją `SQLiteOpenHelper`²². **SQLiteOpenHelper** to klasa pomocnicza, która zarządza tworzeniem i aktualizacją bazy. Aby użyć jej funkcji, tworzymy podklasę i implementujemy metody `onCreate(SQLiteDatabase db)` (tu wykonujemy np. `db.execSQL("CREATE TABLE ...")`) oraz `onUpgrade(...)`. Przykład:

```
public class MyDbHelper extends SQLiteOpenHelper {
    public MyDbHelper(Context context) {
        super(context, "mojaBaza.db", null, 1);
    }
    @Override
    public void onCreate(SQLiteDatabase db) {
        db.execSQL("CREATE TABLE Osoba (id INTEGER PRIMARY KEY, imie TEXT)");
    }
    @Override
    public void onUpgrade(SQLiteDatabase db, int oldV, int newV) {
        db.execSQL("DROP TABLE IF EXISTS Osoba");
        onCreate(db);
    }
}
```

Następnie w `Activity` lub innym miejscu uzyskujemy dostęp do bazy:

```
MyDbHelper dbHelper = new MyDbHelper(this);
SQLiteDatabase db = dbHelper.getWritableDatabase();
ContentValues cv = new ContentValues();
```

```
cv.put("imie", "Anna");
db.insert("Osoba", null, cv);
```

Android domyślnie przechowuje bazę w prywatnym katalogu aplikacji na urządzeniu, więc jest chroniona ²³. Nowoczesne aplikacje mobilne mogą również używać biblioteki **Room** (część Android Jetpack), która dodatkowo upraszcza operacje na bazie (ORM). Jednak do egzaminu wystarczy rozumieć `SQLiteOpenHelper` i podstawowe operacje (`insert`, `query`, `update`, `delete`).

Konfiguracja i bezpieczeństwo: Android umożliwia również zapis do pliku w pamięci wewnętrznej (`openFileOutput`) oraz `SharedPreferences` do przechowywania prostych ustawień. Dostęp do Internetu wymaga dodania uprawnienia w pliku `AndroidManifest.xml` (`<uses-permission android:name="android.permission.INTERNET"/>`).

Wzorce projektowe w aplikacjach mobilnych

W aplikacjach Android często stosuje się podobne wzorce jak w Javie desktopowej/webowej. MVC/MVVM to przykłady architektur (w mobilnym world – Android Studio promuje MVVM z LiveData, ale INF.04 zapewne wystarczy klasyczne MVC). Singleton może być użyty np. do klasy zarządzającej połączeniem do bazy lub konfiguracją. Przykład Singleton w Androidzie: użycie klasy `Application` do trzymania instancji bazy. Factory może się pojawiać przy tworzeniu adapterów do `RecyclerView` itp. Observer – Android ma własne wzorce obserwatorów, np. `LiveData` czy zdarzenia UI (Observer pattern wewnątrz View binding).

Ogólnie wzorce z Java ogólnego zastosowania (Singleton, Factory, Observer, DAO) działają identycznie w kontekście Androida – jedynie sposób integracji z UI jest specyficzny (np. zamiast konsoli mamy ekran dotykowy). W przypadku Androida warto też znać wzorzec **Adapter** (przydatny np. przy `ListView` lub `RecyclerView`) i **ViewHolder** dla optymalizacji list.

Testowanie aplikacji Android

Testy w Androidzie dzielą się na dwa typy: **jednostkowe** (local unit tests) i **instrumentacyjne (UI tests)**. Local tests korzystają z JUnit (działają na maszynie JVM), a instrumentacyjne – z Android Testing Support Library (np. Espresso) i uruchamiane są na emulatorze/urządzeniu. Dla INF.04 wystarczy wspomnieć JUnit do testów logiki (np. metod pomocniczych) oraz testy UI dla sprawdzenia widoków.

Przykładowy test jednostkowy (logika bez UI):

```
public class UtilsTest {
    @Test
    public void testSumuj() {
        assertEquals(10, Utils.sumuj(4, 6));
    }
}
```

Testowanie UI (opcjonalnie): użycie Espresso, np. sprawdzenie, czy po kliknięciu przycisku TextView zmienia tekst. To raczej zaawansowane, ale warto wspomnieć, że w Androidzie testy instrumentacyjne uruchamia się przez klasę `@RunWith(AndroidJUnit4.class)` i adnotację `@Test`.

REST API (Android + sieć)

Aplikacje mobilne często komunikują się z serwerem REST (np. pobieranie danych JSON). W Javie Androida można wykorzystać `URLConnection` lub biblioteki jak **Retrofit**. Przykład użycia `URLConnection` jest podobny jak w zwykłej Javie (z wyjątkiem wywoływania w tle, jak wspomniano). Alternatywnie Android udostępnia `AsyncTask` do asynchronicznego pobierania, lub lepiej – wykorzystać `OkHttp` / `Retrofit`. W INF.04 można opisać to słowami: należy korzystać z wątków/`AsyncTask`, aby pobrać dane JSON z REST API i parsować je przy pomocy np. `JSONObject` / `JSONArray`.

Bluetooth/Media: W mobilnej aplikacji można również dotykać tematów specyficznych dla Androida: wysyłanie SMS, odtwarzanie multimediów (`MediaPlayer`), ale to rzadziej pojawia się w zadaniach INF.04. Główne zagadnienia to UI, bazy danych i REST.

UML i struktura aplikacji Android

Architektura aplikacji mobilnej najczęściej opiera się na warstwach (podobnie jak MVC/MVVM). Dobrze jest przed implementacją narysować diagram przepływu aplikacji (jak przechodzimy między `Activity`) oraz diagram klas modelu danych (np. klasy reprezentujące encje bazy). Przykładowy diagram klas mobilnej aplikacji może zawierać klasy: `User`, `Book`, `DatabaseHelper`, `MainActivity`, `DetailActivity` itp., z relacjami zależności (np. `MainActivity` korzysta z `DatabaseHelper`). Dla UML w Androidzie stosuje się te same notacje: prostokąty klas, strzałki dziedziczenia, itp.

Schematy ekranu: Można również przygotować prosty „diagram ekranu” – schematyczny podział ekranu na widoki (np. Layout XML w formie rysunku). Nie jest to oficjalny diagram UML, ale pomaga w planowaniu UI przed kodowaniem.

Przykłady kodu i zadania egzaminacyjne

Przykład 1: Rejestr książek w Androidzie

Zadanie: Stwórz aplikację mobilną, która pozwala dodawać książki do lokalnej bazy SQLite i wyświetlać ich listę.

Opis rozwiązania: W `MainActivity` umieszczamy pola `EditText` dla tytułu i autora oraz przycisk „Zapisz”. Po kliknięciu zapytanie do `SQLiteOpenHelper.insert()`, który dodaje książkę do tabeli. Lista książek wyświetlana jest w drugim ekranie (`ListActivity` lub `RecyclerView` w tej samej aktywności). Można zastosować `RecyclerView` z `Adapterem`, który pobiera dane przy pomocy kwerendy SQL.

Kod fragmentu zapisu do bazy w `MainActivity`:

```
MyDbHelper dbHelper = new MyDbHelper(this);
SQLiteDatabase db = dbHelper.getWritableDatabase();
ContentValues cv = new ContentValues();
```

```

cv.put("tytul", titleEditText.getText().toString());
cv.put("autor", authorEditText.getText().toString());
long id = db.insert("Ksiazki", null, cv);
if (id != -1) {
    Toast.makeText(this, "Dodano książkę", Toast.LENGTH_SHORT).show();
}
db.close();

```

Analiza: W zadaniu oceniana jest umiejętność tworzenia interfejsu w XML, reagowania na kliknięcia, pracy z bazą SQLite w `Activity` oraz wyświetlania listy (np. w `ListView` lub `RecyclerView`). Typowe pułapki: nie wywołanie `db.close()`, próba operacji I/O na wątku głównym (należy użyć `AsyncTask` lub `ViewModel` z LiveData), zapomnienie o uprawnieniach (choć do bazy lokalnej nie są potrzebne).

Przykład 2: Prosta aplikacja REST

Zadanie: Utwórz aplikację, która pobiera dane pogodowe z zewnętrznego API REST i wyświetla temperaturę dla podanego miasta.

Opis: W `Activity` dodaj `EditText` na nazwę miasta i przycisk „Sprawdź pogodę”. Po naciśnięciu rozpoczyna się żądanie HTTP (np. do serwisu OpenWeatherMap). Odpowiedź (JSON) zawiera pole z temperaturą – wyświetlamy je w `TextView`. Kod pobierający:

```

new Thread(() -> {
    try {
        URL url = new URL("https://api.weather.com/temp?city=" +
        URLEncoder.encode(miasto, "UTF-8"));
        HttpURLConnection conn = (HttpURLConnection) url.openConnection();
        conn.setRequestMethod("GET");
        InputStream in = conn.getInputStream();
        Scanner scanner = new Scanner(in);
        StringBuilder json = new StringBuilder();
        while (scanner.hasNext()) json.append(scanner.nextLine());
        JSONObject obj = new JSONObject(json.toString());
        double temp = obj.getDouble("temperature");
        runOnUiThread(() -> wynikTextView.setText(temp + " °C"));
    } catch (Exception e) {
        e.printStackTrace();
    }
}).start();

```

Analiza: Zadanie sprawdza zdolność do obsługi wątków oraz parse'u JSON. Ważne jest wywołanie `runOnUiThread` (lub użycie `AsyncTask`) aby zaktualizować UI z wątku pobierania. W odpowiedzi JSON należy znaleźć właściwe pola (za pomocą np. `JSONObject`). Błędy: `NetworkOnMainThreadException` przy niewłaściwym zarządzaniu wątkiem, błędne kodowanie URL (należy użyć `URLEncoder`), brak obsługi wyjątków JSON gdy API odpowiada błędem.

Typowe błędy i debugowanie w Androidzie

- `NullPointerException` przy `findViewById` – występuje, gdy wywołamy `findViewById` przed `setContentView` lub dla widoku nieobecnego w aktualnym layoutcie (np. literówka w `@+id`). Debug: upewnij się, że element istnieje w XML i `findViewById` wykonujesz po wczytaniu layoutu.
- `java.lang.RuntimeException: Manifest merger failed` – zwykle konflikt zależności lub duplikat uprawnień. Sprawdź plik `AndroidManifest.xml` i zależności Gradle.
- `ActivityNotFoundException` – próba uruchomienia aktywności, której nie zadeklarowano w `AndroidManifest.xml`.
- **Brak uprawnień** – np. brak `<uses-permission android:name="android.permission.INTERNET"/>` jeśli używamy sieci (aplikacja zablokuje połączenie). Dodaj wymagane uprawnienia.
- **Niewłaściwe działania na wątku głównym** – ANR (Application Not Responding) pojawia się, gdy wczytujesz dużą ilość danych na głównym wątku (np. z bazy lub sieci). Rozwiązanie: użyj `AsyncTask` lub `Thread`.
- **Błędy kompilacji XML** – literówki w nazwach elementów, brakujący `xmlns:android` w nagłówku layoutu, nadmiarowy lub brakujący atrybut układu (`layout_width`, `layout_height` są obowiązkowe).
- **Obiekty Cursors** – zapomnienie zamknięcia `Cursor` po zakończeniu (może wyciek pamięci).

Porada do debugowania: Użyj `Log.d(TAG, "wiadomość")` aby wypisywać komunikaty w Logcat. Ustaw punkty przerwania (breakpoints) i debugger w Android Studio, aby krok po kroku sprawdzić zmienne. W Logcat filtruj po swojej aplikacji (tag lub nazwie pakietu).

Aplikacje okienkowe – JavaFX + Scene Builder

Spis treści

- Tworzenie projektu JavaFX (Scene Builder)
- Pierwszy interfejs użytkownika (FXML, kontrolery)
- Obsługa zdarzeń i baza aplikacji (MVC)
- Style CSS i multimedia w JavaFX
- Bazy danych (SQLite + JavaFX)
- Wzorce projektowe w aplikacjach desktopowych
- Testowanie aplikacji JavaFX
- REST API (komunikacja sieciowa)
- UML i architektura aplikacji JavaFX
- Przykłady kodu i zadania egzaminacyjne
- Typowe błędy i debugowanie w JavaFX

Tworzenie projektu JavaFX (Scene Builder)

JavaFX to biblioteka do tworzenia nowoczesnych GUI w Javie. Scene Builder to narzędzie do graficznego projektowania interfejsu. Aby rozpocząć: zainstaluj Scene Builder (np. od GluonHQ) ²⁴. W IntelliJ IDEA

(wersja 15+) i nowszych wystarczy wskazać ścieżkę do Scene Buildera w ustawieniach (File > Settings > Languages & Frameworks > JavaFX) ²⁵.

Tworzymy nowy projekt JavaFX (w IntelliJ typu JavaFX). Następnie tworzymy plik FXML (np. `MainLayout.fxml`) – w Scene Builderze przeciągamy przyciski, pola itp. i zapisujemy plik. Scene Builder wygeneruje XML z hierarchią kontrolerek ²⁶. W pliku FXML należy powiązać `fx:controller="nazwa.klasy.Kontrolera"`. Następnie tworzymy klasę kontrolera:

```
public class MainController {
    @FXML private Button clickButton;
    @FXML private Label infoLabel;
    public void initialize() {
        clickButton.setOnAction(e -> infoLabel.setText("Kliknięto!"));
    }
}
```

Gdy uruchomimy aplikację, JavaFX odczyta FXML, utworzy scenę i wyświetli okno. Konfiguracja projektu obejmuje dodanie bibliotek JavaFX (od Javy 11 nie są domyślnie w JDK, trzeba dodać moduły lub zależności Maven/Gradle).

Pierwszy interfejs użytkownika (FXML, kontrolery)

Podstawowe elementy GUI: `Stage` (okno), `Scene` (obszar sceny) i drzewo węzłów (kontrolerek: `Button`, `Label`, `TextField`, `ListView` itd.). Można pisać interfejs ręcznie w kodzie, ale FXML + Scene Builder przyspiesza pracę ²⁶.

Proces: 1. W `start(Stage primaryStage)` w głównej klasie ładujemy FXML:

```
Parent root = FXMLLoader.load(getClass().getResource("MainLayout.fxml"));
primaryStage.setScene(new Scene(root));
primaryStage.setTitle("Moja Aplikacja FX");
primaryStage.show();
```

2. FXML definiuje widoki, a kontroler (`@FXML`) łączy się z kodem Javy. W pliku kontrolera tworzymy metody `init` i akcje kliknięć.

JavaFX wspiera `binding` właściwości, można dynamicznie wiązać wartości (np. `textField.textProperty().bind(model.observableProperty())`). Dzięki temu aktualizacja modelu automatycznie odświeża widok.

Obsługa zdarzeń i baza aplikacji (MVC)

W aplikacji JavaFX warto wyraźnie oddzielić warstwę GUI (View), logikę (Model) i pośrednika (Controller) – czyli zastosować wzorzec **MVC**. Kontroler przejmuje zdarzenia (np. kliknięcie) i modyfikuje model lub zleca

mu obliczenia, a model przekazuje dane do widoku. Przykład: model `Osoba` z właściwościami `name`, które można dowiązać do `Label`.

Obsługa zdarzeń w JavaFX odbywa się np. poprzez `setOnAction` w kontrolerze lub deklarację `onAction="#metoda"` w FXML, która odwołuje się do publicznej metody w kontrolerze. Struktura aplikacji JavaFX to zazwyczaj: - Klasa główna (rozszerzająca `Application`), - Kilka plików FXML i odpowiadających im klas kontrolerów, - Klasy modelu (JavaBeans, często z właściwościami JavaFX - `StringProperty`, `IntegerProperty` dla automatycznego odświeżania UI).

Style CSS i multimedia w JavaFX

JavaFX pozwala na stylowanie kontrolki za pomocą CSS. Możemy dołączyć plik `.css` i przypisać klasy, np. `my-button { -fx-background-color: blue; }`. Style ustawiamy w FXML lub w kodzie (np. `button.getStyleClass().add("my-button")`).

Dodatkowo JavaFX ma wbudowane wsparcie multimediów: odtwarzanie audio (`MediaPlayer`, `AudioClip`) oraz wideo (`MediaView`). Istnieje też komponent `WebView` z silnikiem WebKit do wyświetlania HTML/JS.

Bazy danych (SQLite + JavaFX)

Aplikacje desktopowe JavaFX również mogą wykorzystywać bazy danych. Przykładowo, można użyć **SQLite** tak jak w konsolowej Javie (dodając sterownik JDBC). Kod dodawania do bazy jest identyczny. Warto wykorzystać DAO, aby oddzielić JavaFX od logiki bazy.

Przykład: metoda w klasie DAO `public List<Ksiazka> pobierzWszystkie();` zwraca listę książek z bazy. Następnie w kontrolerze możemy dołączyć te dane do `TableView` (tablica z kolumnami). JavaFX ma klasę `TableView<T>` oraz `TableColumn` - pozwala automatycznie wyświetlić pola obiektów (za pomocą `PropertyValueFactory` lub `cellValueFactory`). Warto uczynić model (np. `Ksiazka`) klasą `Observable`, aby zmiany odzwierciedlały się w UI.

Schemat:

```
TableView<Ksiazka> table = new TableView<>();
TableColumn<Ksiazka, String> colTytul = new TableColumn<>("Tytuł");
colTytul.setCellValueFactory(new PropertyValueFactory<>("tytul"));
table.getColumns().add(colTytul);
// analogicznie inne kolumny

List<Ksiazka> lista = dao.pobierzWszystkie();
table.setItems(FXCollections.observableArrayList(lista));
```

Wzorce projektowe w aplikacjach desktopowych

W aplikacjach JavaFX także korzystamy z ogólnych wzorców: - **MVC**: opisany powyżej, może być nawet rozszerzony do MVP lub MVVM (z `ViewModel` i `Bindings`).

- **Singleton**: np. do zarządzania instancją bazy danych lub konfiguracją.

- **Observer**: JavaFX sam używa wzorca Observer (właściwości `Property` notyfikują listenerów). Przykładem może być `ChangeListener`.

- **Decorator**: pojawia się przy strumieniach (jak w konsolowej Javie) i też przy komponowaniu kontroltek (`BorderPane` dekoruje `Center` inną kontrolką).

- **Factory**: tworzenie złożonych kontroltek (np. fabryka ustawiająca wygląd `TableCell`).

Niektóre wzorce, jak np. Adapter, mogą być użyte przy integracji z innymi systemami lub starym kodem (np. adaptacja klasy `LocalDate` do formatu SQL).

Testowanie aplikacji JavaFX

Testy jednostkowe w JavaFX pisze się tak samo jak w Javie („czysta” logika modelu). Do testowania GUI można użyć biblioteki **TestFX** – pozwala na symulację kliknięć w kontrolki. Przykład prostego testu TestFX:

```
import org.testfx.framework.junit5.ApplicationTest;
import static org.junit.jupiter.api.Assertions.*;
public class MainControllerTest extends ApplicationTest {
    @Override
    public void start(Stage stage) throws Exception {
        Parent root =
FXMLLoader.load(getClass().getResource("MainLayout.fxml"));
        stage.setScene(new Scene(root));
        stage.show();
    }
    @Test
    public void testButtonKlik() {
        clickOn("#clickButton");
        Label label = lookup("#infoLabel").query();
        assertEquals("Kliknięto!", label.getText());
    }
}
```

Dzięki TestFX można sprawdzić, czy interfejs reaguje poprawnie na zdarzenia. Jednak w INF.04 wystarczy wiedzieć, że testowanie możliwe jest poprzez narzędzia JUnit (logika) i TestFX (UI).

REST API (komunikacja sieciowa)

Jeśli aplikacja JavaFX ma pobierać dane z sieci, można zastosować `URLConnection` podobnie jak w konsolowej Javie. Należy tworzyć wątki asynchroniczne (np. `Task<Void>`) do pobierania, ponieważ GUI nie może być blokowane. Po pobraniu i sparsowaniu JSON możemy zaktualizować interfejs przez `Platform.runLater(() -> { ... })`. Przykład schematu:

```

Task<Void> task = new Task<>() {
    @Override
    protected Void call() throws Exception {
        URL url = new URL("https://api.example.com/dane");
        HttpURLConnection conn = (HttpURLConnection) url.openConnection();
        InputStream in = conn.getInputStream();
        String json = new Scanner(in).useDelimiter("\\A").next();
        JSONObject obj = new JSONObject(json);
        String wartosc = obj.getString("pole");
        Platform.runLater(() -> {
            labelValue.setText(wartosc);
        });
        return null;
    }
};
new Thread(task).start();

```

UML i architektura aplikacji JavaFX

Podobnie jak w innych środowiskach, warto przygotować **diagram klas** i ewentualnie **diagram sekwencji** dla interakcji (np. opis przepływu akcji między GUI a bazą). Klasy: `MainController`, `DataModel`, `Repository` itp. Często używane jest pojęcie *architektury warstwowej* – oddzielenie logiki aplikacji (np. klasy odpowiadające za obliczenia lub dostęp do danych) od GUI. Diagram klas można narysować przy pomocy narzędzi (np. PlantUML, narzędzi IDE lub ręcznie).

UML w praktyce: UML pozwala zobrazować strukturę aplikacji przed kodowaniem. Na przykład diagram klas pokazujący `Produkt`, `Koszyk` itp. w aplikacji sklepowej znacznie ułatwia zrozumienie zależności.

Przykłady kodu i zadania egzaminacyjne

Przykład 1: Kalkulator GUI

Zadanie: Napisz aplikację JavaFX, która działa jak prosty kalkulator (dodawanie/odejmowanie dwóch liczb).

Opis: W GUI mamy dwa pola `TextField` do wpisania liczb oraz dwa przyciski „+” i „-” i pole `Label` do wyświetlania wyniku. Po kliknięciu „+” suma liczb pokazuje się w labelu. Analogicznie dla „-”.

Fragment kontrolera:

```

public class CalculatorController {
    @FXML private TextField txtA, txtB;
    @FXML private Label lblResult;

    @FXML
    public void initialize() {

```

```

        // ustawienia początkowe
    }

    @FXML
    private void onAdd() {
        try {
            double a = Double.parseDouble(txtA.getText());
            double b = Double.parseDouble(txtB.getText());
            lblResult.setText(String.valueOf(a + b));
        } catch (NumberFormatException e) {
            lblResult.setText("Nieprawidłowy format liczby");
        }
    }

    @FXML
    private void onSubtract() {
        try {
            double a = Double.parseDouble(txtA.getText());
            double b = Double.parseDouble(txtB.getText());
            lblResult.setText(String.valueOf(a - b));
        } catch (NumberFormatException e) {
            lblResult.setText("Nieprawidłowy format liczby");
        }
    }
}

```

Analiza: To zadanie sprawdza podstawową integrację GUI z kodem, obsługę wyjątków (`NumberFormatException` przy niepoprawnym wpisie), a także prawidłową obsługę zdarzeń (metody `onAdd()`, `onSubtract()` oznaczone w FXML jako `onAction`). W raporcie egzaminacyjnym opiszemy strukturę FXML (np. `fx:id="txtA"`, `onAction="#onAdd"`).

Przykład 2: Aplikacja z bazą danych

Zadanie: Utwórz aplikację JavaFX do zarządzania listą studentów. Powinna umożliwiać dodawanie studenta (imie, nazwisko, nr indeksu) do bazy SQLite i wyświetlanie ich w `TableView`.

Opis: W FXML dodajemy pola `TextField` do wpisania danych studenta oraz przycisk „Dodaj”. Po wciśnięciu pobieramy dane, wywołujemy DAO, a następnie odświeżamy tabelę. `TableView<Student>` ma kolumny `Imię`, `Nazwisko`, `NrIndeksu`.

Fragment kodu zapisu:

```

public class StudentsController {
    @FXML private TextField txtImie, txtNazwisko, txtIndeks;
    @FXML private TableView<Student> tableStudents;
    @FXML private TableColumn<Student, String> colImie, colNazwisko, colIndeks;
}

```

```

private StudentDAO dao = new StudentDAO();

@FXML
public void initialize() {
    colImie.setCellValueFactory(new PropertyValueFactory<>("imie"));
    colNazwisko.setCellValueFactory(new PropertyValueFactory<>("nazwisko"));
    colIndeks.setCellValueFactory(new PropertyValueFactory<>("nrIndeksu"));
    refreshTable();
}

@FXML
private void onDodaj() {
    String imie = txtImie.getText();
    String nazwisko = txtNazwisko.getText();
    String indeks = txtIndeks.getText();
    dao.insertStudent(new Student(imie, nazwisko, indeks));
    refreshTable();
}

private void refreshTable() {
    List<Student> lista = dao.getAllStudents();
    tableStudents.setItems(FXCollections.observableArrayList(lista));
}
}

```

Analiza: Kluczowe aspekty: konfiguracja `TableView` (wymaga prawidłowego `fx:id` w FXML i metod `PropertyValueFactory`) oraz integracja z bazą. DAO zajmuje się komunikacją z SQLite, a kontroler tylko wywołuje metody DAO i odświeża widok. Błędy: brak `txtNazwisko.getText()` itp., lub wywołanie `refreshTable()` bez pobrania nowej instancji listy. Pamiętaj, że `TableView` musi działać na liście obserwowalnej (tutaj `FXCollections.observableArrayList()`).

Typowe błędy i debugowanie w JavaFX

- `NullPointerException` **na kontrolerze:** Zwykle przez niezgodność `fx:id` w FXML i nazw w kontrolerze albo brak adnotacji `@FXML`. Upewnij się, że `fx:id="myField"` i w kontrolerze `@FXML private TextField myField;` pasuje.
- `java.lang.UnsupportedOperationException: Not allowed on FX application thread` – występuje, gdy próbujesz wykonywać operacje długotrwałe (np. `Thread.sleep()`) w wątku GUI. Rozwiązanie: użyj `Task` / `Service`.
- **Brak ścieżki do FXML:** `FXMLLoader.load(...)` zwraca błąd jeśli `MainLayout.fxml` nie został znaleziony (sprawdź ścieżkę i umieszczenie pliku w zasobach).
- **Błędy CSS:** niepoprawna składnia w pliku `.css` może spowodować, że style nie są aplikowane – sprawdź nazwę klasy w `getStyleClass()` i zapis w CSS.

Kroki debugowania: Podobnie jak w Javie konsolowej, warto użyć `System.out.println` lub `Logger` w metodzie `initialize()` i handlerach, aby upewnić się, że są wywoływane.

IntelliJ umożliwia debugowanie również aplikacji JavaFX – można postawić breakpoint w kontrolerze i prześledzić wykonanie kodu.

Aplikacje webowe – React + Vite

Spis treści

- Tworzenie projektu (Vite + React)
- Podstawy React (JSX, komponenty, props i state)
- Interakcja z użytkownikiem (formularze, zdarzenia)
- Stan aplikacji i zarządzanie danymi
- Wysyłanie żądań do REST API (fetch / axios)
- Routing (np. React Router)
- Wzorce architektoniczne (komponenty funkcyjne vs. klasowe, MVC w React)
- Testowanie aplikacji React (Jest, React Testing Library)
- DevOps (Vite konfiguracja i budowanie)
- Przykłady kodu i zadania egzaminacyjne
- Typowe błędy i debugowanie w React

Tworzenie projektu (Vite + React)

W nowoczesnym środowisku webowym zamiast klasycznych serwerów używa się **narzędzi budujących (bundlerów)**. Vite to szybki bundler stworzony z myślą o React (i innych frameworkach). Aby zainicjować nowy projekt React z Vite, używamy polecenia:

```
npm create vite@latest my-app --template react
```

To polecenie utworzy katalog `my-app` z szablonem React i Vite ²⁷. Następnie wchodzimy do katalogu i instalujemy zależności (`npm install`). W pliku `package.json` znajdziemy skrypty: `npm run dev` uruchamia serwer deweloperski (na `http://localhost:3000` domyślnie), a `npm run build` tworzy produkcyjną wersję.

Podstawy React (JSX, komponenty, props i state)

React to biblioteka do budowania interfejsu użytkownika na zasadzie **komponentów**. Komponenty mogą być klasowe (dziedziczą po `React.Component`) lub funkcyjne (z funkcją zwracającą JSX). Obecnie zaleca się komponenty funkcyjne z **hookami**.

Przykład komponentu funkcyjnego (JSX):

```
function Hello(props) {  
  return <h1>Witaj, {props.name}</h1>;  
}
```

```
// Użycie w innym komponencie:  
<Hello name="Alice" />
```

JSX pozwala pisać HTML-owym stylu kod w JavaScript. Kompilowany jest do `React.createElement(...)`. `props` to właściwości przekazywane do komponentu (tzw. *parametry*). Dodatkowo komponent może mieć **stan** (`state`). Przy użyciu hooka `useState` definiujemy stan wewnętrzny:

```
import { useState } from 'react';  
  
function Counter() {  
  const [count, setCount] = useState(0);  
  return (  
    <div>  
      <p>Liczba kliknięć: {count}</p>  
      <button onClick={() => setCount(count + 1)}>Kliknij</button>  
    </div>  
  );  
}
```

Tutaj `count` to zmienna stanu, a `setCount` funkcja do jej aktualizacji. Kliknięcie przycisku zwiększa licznik. React zapewnia **jednokierunkowy przepływ danych**: dane (props) płyną z góry (rodzic -> dzieci), a zmiany stanu odbywają się przez `useState` lub inne hooki.

Interakcja z użytkownikiem (formularze, zdarzenia)

Reagowanie na zdarzenia w React to np. użycie `onClick`, `onChange` itp. Przykład prostego formularza:

```
function Form() {  
  const [text, setText] = useState("");  
  function handleSubmit(e) {  
    e.preventDefault();  
    alert("Wysłano: " + text);  
  }  
  return (  
    <form onSubmit={handleSubmit}>  
      <input value={text} onChange={e => setText(e.target.value)} />  
      <button type="submit">Wyślij</button>  
    </form>  
  );  
}
```

Elementy formularza (`<input>`, `<select>`) często kontrolujemy poprzez stan (tzw. **formularze kontrolowane**). Dzięki `value={text}` i `onChange` React utrzymuje źródło prawdy stanu w komponencie.

Stan aplikacji i zarządzanie danymi

W większych aplikacjach stan aplikacji może być złożony. React oferuje mechanizmy: `useState`, `useReducer`, `Context` (globalny kontekst). Na potrzeby INF.04 wystarczy znać `useState`. Dla routingu między „stronami” można użyć biblioteki **React Router** – definiuje komponent `<Router>` i `<Route>` dla różnych ścieżek URL.

Jeśli potrzebna jest globalna pamięć stanu (np. zalogowany użytkownik dostępny we wszystkich komponentach), stosuje się `React.createContext` i `useContext`, lub zewnętrzne biblioteki jak Redux (raczej poza zakresem INF.04).

Wysyłanie żądań do REST API (fetch / axios)

Do komunikacji z serwerem używamy w React funkcji `fetch` (lub bibliotek jak `axios`). Przykład pobrania danych w hooku `useEffect`:

```
import { useState, useEffect } from 'react';

function Weather() {
  const [temp, setTemp] = useState(null);

  useEffect(() => {
    fetch("https://api.example.com/weather?city=Warsaw")
      .then(response => response.json())
      .then(data => setTemp(data.temperature))
      .catch(error => console.error(error));
  }, []); // [] oznacza wywołaj tylko raz po zamontowaniu

  return <div>Temperatura: {temp ? temp + " °C" : "Ładowanie..."}</div>;
}
```

Tutaj `fetch` zwraca obietnicę, którą obsługujemy `.then()`. Po otrzymaniu JSON aktualizujemy stan `temp` i React automatycznie odświeża widok. Należy pamiętać, że `fetch` domyślnie nie zgłasza błędów dla statusów HTTP poza 200–299, więc warto sprawdzić `response.ok`. Obsługa żądań w React jest asynchroniczna, ale dzięki `.then` lub `async/await` kod jest czytelny.

Routing (React Router)

Typowe aplikacje webowe mają kilka „stron” (widoków). W React do tego służy **React Router**. Przykład:

```
import { BrowserRouter, Routes, Route } from 'react-router-dom';

function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/" element={<HomePage />} />
        <Route path="/about" element={<AboutPage />} />
      </Routes>
    </BrowserRouter>
  );
}
```

W `<HomePage />` i `<AboutPage />` umieszczamy różne komponenty. Użytkownik może nawigować (np. `<Link to="/about">O nas</Link>`). Routing pozwala utrzymywać pojedynczą stronę HTML, a tylko zmieniać komponenty React w obszarze aplikacji.

Wzorce architektoniczne (komponenty funkcyjne vs. klasowe, MVC w React)

React nie wymusza klasycznego MVC, ale można myśleć o `App` jako kontrolerze, `Model` w stanie (lub zewnętrznym API) i `View` jako komponentach renderujących HTML. Klasyczne MVC w czystej formie nie występuje, bo cały interfejs jest definicją w komponentach.

Warto jednak zauważyć, że komponent React może pełnić rolę *widoku+kontrolera* (obsługuje zdarzenia i wyświetla UI). **Komponenty klasowe** (przed React 16.8) miały wewnętrzny stan (`this.state`) i metody cyklu życia (`componentDidMount`). Obecnie zaleca się **komponenty funkcyjne** i hooki. Mimo nazewnictwa w INF.04 nie używamy tu `super` czy dziedziczenia (poza samego `React.Component`). Ważna jest idea ponownego użycia komponentów – dzielenie UI na mniejsze części (np. komponent `Button`, `UserCard`, `BookList`).

Testowanie aplikacji React (Jest, React Testing Library)

React jest często testowany z użyciem **Jest** (framework testowy JS) i **React Testing Library**. Testy pisze się np.:

```
import { render, screen, fireEvent } from '@testing-library/react';
import { Hello } from './Hello';

test('wyświetla powitanie', () => {
  render(<Hello name="Jan" />);
  expect(screen.getByText(/Witaj, Jan/i)).toBeInTheDocument();
});
```

Jest pozwala testować logikę oraz snapshoty komponentów. React Testing Library skupia się na testowaniu aplikacji tak, jak użytkownik. Dzięki nim sprawdzamy, czy UI renderuje się poprawnie i reaguje na zdarzenia.

DevOps (Vite konfiguracja i budowanie)

Vite zapewnia szybką pracę dzięki natywnemu ES Module i hot-reloadowi. W pliku `vite.config.js` możemy dostosować port, proxy, aliasy. Dla budowania aplikacji do produkcji używamy `npm run build` - generuje zoptymalizowane pliki w folderze `dist`, które następnie umieszczamy na serwerze WWW. W INF. 04 wystarczy znać podstawowe polecenia (`dev`, `build`) i umiejętność odtworzenia prostego skryptu lub konfiguracji.

Przykłady kodu i zadania egzaminacyjne

Przykład 1: Kalkulator w React

Zadanie: Zaimplementuj prosty kalkulator w React – dwa inputy `number` i przyciski dla operacji `+` i `-`. Wynik wyświetlany jest na stronie.

Rozwiązanie: Komponent może wyglądać tak:

```
import { useState } from 'react';

function Calculator() {
  const [a, setA] = useState(0);
  const [b, setB] = useState(0);
  const [result, setResult] = useState(0);

  return (
    <div>
      <input type="number" value={a} onChange={e => setA(+e.target.value)} />
      <input type="number" value={b} onChange={e => setB(+e.target.value)} />
      <button onClick={() => setResult(a + b)}>+</button>
      <button onClick={() => setResult(a - b)}>-</button>
      <p>Wynik: {result}</p>
    </div>
  );
}
```

Analiza: Zwróć uwagę na `value={a}` i `onChange`, to tzw. *kontrolowane inputy*. Operator `+e.target.value` konwertuje string na liczbę. Przycisk `+` dodaje wartości, `-` odejmuje, a wynik przechowujemy w stanie `result`. Błąd: bez `value` i `onChange` inputy nie reagują na wpisywanie. Brak `type="number"` spowoduje, że wartość jest stringiem i dodawanie by znaczyło konkatencję tekstu.

Przykład 2: Lista z API

Zadanie: Utwórz komponent React, który wyświetla listę postów pobranych z adresu `https://jsonplaceholder.typicode.com/posts`.

Rozwiązanie: Możemy użyć hooka `useEffect`:

```
import { useState, useEffect } from 'react';

function PostsList() {
  const [posts, setPosts] = useState([]);

  useEffect(() => {
    fetch("https://jsonplaceholder.typicode.com/posts")
      .then(res => res.json())
      .then(data => setPosts(data))
      .catch(err => console.error(err));
  }, []);

  return (
    <ul>
      {posts.map(post => (
        <li key={post.id}>{post.title}</li>
      ))}
    </ul>
  );
}
```

Analiza: Po zamontowaniu komponentu (pusty `[]` w `useEffect` – wywołanie raz) pobieramy dane, zamieniamy na JSON i zapisujemy w stanie. W `render` mapujemy po tablicy `posts` i wyświetlamy tytuły w listach. Ważne jest dodanie atrybutu `key={post.id}` dla wydajności Reacta. Błędy: brak `key` spowoduje ostrzeżenie. `fetch` jest asynchroniczny, upewnij się, że nie renderujesz stanu zanim się załaduje (tutaj początkowo jest pusta tablica, więc lista będzie pusta a potem się zaktualizuje).

Typowe błędy i debugowanie w React

- **Brak `value` / `onChange` w `<input>` (formularz kontrolowany):** input nie aktualizuje się poprawnie lub nie można wpisać tekstu. Rozwiązanie: zastosuj `useState` i powiąż `value`.
- **„Unexpected token” w JSX:** zapomniałeś dołączyć Babel lub plik ma błąd składni (np. JSX musi być zamknięty jedną dużą linijką). Upewnij się, że masz poprawne ustawienia w Vite (domyślnie wspiera JSX).
- **Nieprawidłowe odwołanie do stanu (np. `setCount(count++)` zamiast `setCount(count + 1)`):** `count++` zwraca wartość *przed* inkrementacją, co może prowadzić do błędnego wyniku. W React zawsze używaj `setCount(count + 1)`.
- **Zapomnienie `key` w `.map()` listy:** React daje ostrzeżenie, że lista elementów powinna mieć `key` dla identyfikacji (najczęściej `item.id`).
- **Nieprawidłowe `import`:** w React nazwy importów muszą być poprawne (`import React from 'react'` nie jest już wymagane w nowych wersjach, ale np. `import Component from './MyComponent'` musi mieć prawidłową ścieżkę).
- **CORS (Cross-Origin):** gdy próbujesz fetchować z innego źródła, serwer może zablokować zapytania. W dev-mode Vite można skonfigurować `proxy` lub inny serwer musi udostępnić odpowiednie nagłówki CORS.

- **Debugowanie:** W Chrome/Firefox otwórz DevTools (F12) i sprawdź konsolę – React wyświetli błędy i ostrzeżenia (np. brak `key`). Możesz także użyć rozszerzenia React Developer Tools do inspekcji struktury komponentów i stanu.

Źródła React/Vite: Oficjalna strona React (<https://react.dev>) oraz Vite (<https://vite.dev>) zawierają dokumentację i tutoriale startowe ²⁷.

Podsumowanie

Powyższe materiały obejmują kluczowe zagadnienia INF.04 dla różnych środowisk programistycznych: konsolowego (Java), mobilnego (Android), desktopowego (JavaFX) i webowego (React/Vite). Każdy dział zawiera teorię, przykłady kodu, typowe błędy i propozycje zadań egzaminacyjnych. Wszystkie omawiane koncepcje – OOP, dziedziczenie (`this`, `super`), interfejsy, wyjątki, kolekcje, pliki/strumienie, wzorce projektowe, testowanie, REST API, bazy danych (JDBC/SQLite), UML/MVC – są ważne na egzaminie INF.04. Z pomocą powyższych materiałów uczeń może przybliżyć strukturę każdego środowiska, zrozumieć potrzebną teorię oraz przećwiczyć praktyczne zadania z kodem i analizą.

Literatura i źródła:

- Dokumentacja JetBrains IntelliJ IDEA – tworzenie projektów Java ¹ ².
- Oficjalny przewodnik tworzenia nowego projektu Android Studio ²¹.
- Poradnik JavaFX i Scene Builder (JavaStart) ²⁴ ²⁵.
- Materiały React – tworzenie aplikacji z Vite ²⁷.
- Kursy i tutoriale Java – dziedziczenie, interfejsy (JavaStart) ²⁸ ⁵.
- Kursy Java – kolekcje (NullPointerException.pl) ⁷ ⁸.
- Tutoriale Java – obsługa plików/strumieni (developeronthego.pl) ¹⁰ ¹¹.
- Tutorial Java – wyjątki (JavaStart) ⁶.
- Wzorce projektowe – artykuł (oig.edu.pl) ¹³ ¹⁴.
- Wikipedia – wzorzec DAO ¹⁵.
- Artykuły o testowaniu (JavaStart) ¹⁶.
- Dokumentacja Android – SQLite i SQLiteOpenHelper ²².
- Tutorial SQLite w Javie (JavaStart) ¹⁹ ¹⁸.
- Poradniki UML – definicja UML (samouczekprogramisty.pl) ²⁰.
- Oficjalna dokumentacja Vite i React (react.dev).

Każdy z powyższych fragmentów pochodzi z autorytatywnych źródeł i tutoriali (zachowano oryginalne cytowania). Źródła ilustrują najważniejsze koncepcje i ułatwiają naukę.

¹ ² Create your first Java application | IntelliJ IDEA Documentation
<https://www.jetbrains.com/help/idea/creating-and-running-your-first-java-application.html>

³ Dziedziczenie cd. i konstrukcja `super()` - JavaStart
<https://javastart.pl/baza-wiedzy/programowanie-obiektowe/dziedziczenie-i-konstrukcja-super>

⁴ Parametry i słowo kluczowe `this` - JavaStart
<https://javastart.pl/baza-wiedzy/programowanie-obiektowe/parametry-i-slowo-kluczowe-this>

5 28 Pytania rekrutacyjne - czym różni się klasa abstrakcyjna od interfejsu? - NullPointerException.pl

<https://nullpointerexception.pl/pytania-rekrutacyjne-czym-rozni-sie-klasa-abstrakcyjna-od-interfejsu/>

6 Wyjątki - blok try catch - JavaStart

<https://javastart.pl/baza-wiedzy/programowanie-objektowe/wyjatki-blok-try-catch>

7 8 9 Kurs Java dla początkujących - #4 Kolekcje - NullPointerException.pl

<https://nullpointerexception.pl/kurs-java-dla-poczatkujacych-4-kolekcje/>

10 11 Java #40: strumień wejścia/wyjścia

<https://developeronthego.pl/java-strumienie-wejscia-wyjscia/>

12 13 14 Wzorce projektowe w praktyce: Singleton, Factory, Observer, MVC

<https://oig.edu.pl/blog/wzorce-projektowe-w-praktyce-singleton-factory-observer-mvc/>

15 Data Access Object – Wikipedia, wolna encyklopedia

https://pl.wikipedia.org/wiki/Data_Access_Object

16 JUnit - Testy jednostkowe w Javie - JavaStart

<https://javastart.pl/baza-wiedzy/testowanie-jednostkowe/junit>

17 How To Use Java HttpURLConnection for HTTP GET and POST Requests | DigitalOcean

<https://www.digitalocean.com/community/tutorials/java-httpurlconnection-example-java-http-request-get-post>

18 19 Bazy danych SQLite w Javie - JavaStart

<https://javastart.pl/baza-wiedzy/java-zaawansowane/bazy-danych-sqlite-w-javie>

20 Podstawy UML - Samouczek Programisty

<https://www.samouczekprogramisty.pl/podstawy-uml/>

21 Create a project | Android Studio | Android Developers

<https://developer.android.com/studio/projects/create-project>

22 23 Zapisz dane przy użyciu SQLite | App data and files | Android Developers

<https://developer.android.com/training/data-storage/sqlite?hl=pl>

24 25 26 JavaFX - JavaStart

<https://javastart.pl/baza-wiedzy/frameworki/javafx>

27 Build a React app from Scratch – React

<https://react.dev/learn/build-a-react-app-from-scratch>